IY3660 – Applications of Cryptography

Week 9 Lecture
Public-Key Encryption Part 2

Elizabeth Quaglia
Information Security Group

Quiz time!

In today's lecture

- The Discrete Logarithm setting and underlying maths
- Diffie-Hellman key exchange
- PKE from discrete logs
- Diffie-Hellman Integrated Encryption Scheme (DHIES)
- Elliptic curve cryptography

The Discrete Logarithm setting

The discrete logarithm setting

- Recall that public key schemes are constructed based on algebraic operations
- The private key operation can be seen as a trapdoor operation
- The trapdoor operation is usually based on mathematical (hard) problems
  - In **RSA**, the public key is (e, N) where N is the product of two primes, p,q
  - The private key is an integer d such that $de = 1 \bmod (p-1)(q-1)$
  - Security depends on hardness of factoring and related problems
- Solving **discrete logarithms** is another example of a mathematical hard problem
- We will see how this setting supports
  - Diffie-Hellman key exchange
  - Public-key encryption
  - Digital signatures

Modular arithmetic over finite fields

- Recall we define the set of integers modulo a (positive) integer n as

$$Z_n = \{ 0 , 1, 2, \ldots, n-1\}$$

with addition, multiplication, exponentation, operations all performed modulo n:

$a + b = c \bmod n$, where c is the remainder of $(a+b)$ divided by n

$a \cdot b = d \bmod n$, where d is the remainder of $(a \cdot b)$ divided by n

$a^k = b \bmod n$, where b is the remainder of $(a \,.\, a \,.\, \ldots \,.\, a)$ [k times] divided by n

- If the modulus is a prime number p, all non-zero elements of Z_p (denoted by Z_p^*) have a multiplicative inverse:
  - For every a in $Z_p^* = \{1,2,\ldots,p-1\}$ there is a unique a^{-1} such that $a \,.\, a^{-1} = 1 \bmod p$
  - You can divide by a non-zero a in Z_p , as it is the case for the rational and real numbers
- We say that **Z_p , for p prime, is a (finite) field**

A bit more algebra

- Let's consider the set $Z_p^* = \{1,2,\ldots,p-1\}$, for p prime
- Let's further restrict to the multiplication operation only
- We will denote Z_p^* as G
- G under the multiplication operation is an example of a group
- The order of a (finite) group is its number of elements
  - In this case the order of G is $p-1$
- Fact: if a group G has order r, then for every $g \in G$, we have $g^r = 1$
  - In this case $g^{p-1} = 1 \bmod p$ for all elements in Z_p^*

Even more algebra…

- As before let $G = Z_p^* = \{1,2,\ldots,p-1\}$, for p prime, under multiplication
- Fact: there is an element g of $G = Z_p^*$ such that all h in G can be expressed as:
  $h = g^k \bmod p$ for some k between 1 and $p-1$.
- G is said to be a cyclic group, and g is a generator *of G*
  - For example, if $p=7$, we have $G = Z_7^* = \{\mathbf{1,2,3,4,5,6}\}$ and $\mathbf{3}$ is a generator of G
    - $3^1 = \mathbf{3} \bmod 7$, $3^2 = 9 = \mathbf{2} \bmod 7$, $3^3 = 27 = \mathbf{6} \bmod 7$,
    - $3^4 = (3^2)^2 = \mathbf{4} \bmod 7$, $3^5 = (3^4).3 = 12 \bmod 7 = \mathbf{5} \bmod 7$ $3^6 = (3^5).3 = 15 \bmod 7 = \mathbf{1} \bmod 7$
    - Note that $3^7 = (3^6).3 = (1).3 \bmod 7 = \mathbf{3} \bmod 7$, and so further powers just run through the same cycle…

Notes:

- $G = Z_p^*$ is an example of a group
  - Groups arise elsewhere in cryptography, and in many, many more other contexts in mathematics and the sciences
- Not all groups are cyclic
- Even when they are, the generator is not necessarily unique
  - E.g., 5 is also a generator of Z_7^*
- if G has a subset H which is also a group, then H is said to be a subgroup of G
  - E.g., H = {**2**, **4**, **1**} is a cyclic subgroup of G = Z_7^* of order 3 generated by the element **2**

The discrete logarithm problem (DLP)

- Let $G = \{g, g^2, g^3, \ldots g^r\}$ be a *cyclic group of order r generated by g*
  - for example, $G = Z_p^*$ for p prime, of order $r = p-1$
- we saw that $g^r = 1$, and will thus write $g^r = g^0$ and take $G = \{g^0, g^1, g^2, g^3, \ldots g^{r-1}\}$ to represent the elements of G

The discrete logarithm problem in a cyclic group G of order r:

Given a randomly selected element $y = g^x$ of $G = \{g^0, g^1, g^2, g^3, \ldots g^{r-1}\}$, **find *x***

- In other words, given y such that $y = g^x$ for some x in [0,r-1], find x
- Think of x as being the *logarithm* of g^x mod p when for the base g
  - Recall that, over positive real numbers, if $y = a^x$, then $x = \log_a(y)$

DLP hardness

- Over the real numbers, computing logarithms is easy
- Over arbitrary finite groups, computing discrete logarithms is (generally) hard
  - DLP is not hard in every finite cyclic group, but it is the case for groups used in cryptography
- Analogy (RSA): computing e^{th}-*roots* over the reals vs. over integers mod $N=p.q$
- Naïve algorithm: keep trying x in [0,r-1] until you have $g^x = h$
  - Applies to any finite group
  - Time complexity: O(r), where r is the order of G
    - i.e., exponential in the bit-size of r
- We can do better than the naïve algorithm!

Solving the DLP

- The DLP has received intense analysis from mathematicians and computer scientists
  - Several different algorithms exist, but they are beyond the scope of this course
    - If you are curious, check the *Further Reading & Research* section
- This is an active area of research, with reasonably recent breakthroughs in related settings

<u>Algorithms for solving the Discrete Log problem:</u>

- the Pollard's rho (ρ) algorithm and its variants:
  - Can be applied to every group
  - Complexity: $O(\sqrt{r})$, where r is the order of the group
    - Still exponential on the number of bits of r
- The Index Calculus algorithm:
  - Dedicated algorithm for Z_p^* (p prime)
  - A DL version of the NFS algorithm (for factorisation)
  - Complexity of the form: $\exp[(1+o(1)).(32/9)^{1/3} (\log p)^{1/3}(\log\log p)^{2/3}]$
    - sub-exponential (but super-polynomial) on the bit size of prime p

Diffie-Hellman key exchange

Diffie-Hellman Key Exchange scheme

- Public-key method for agreeing on a shared secret
- Introduced in a seminal paper from 1976 that launched public key cryptography
  - But refer to reports of early work done inside GCHQ
- **NOT** Public-Key Encryption, but almost there!
- We will introduce DH in the Discrete Logarithm setting over the (multiplicative) group of (non-zero) integers modulo a prime p
  - Note: $p = k.q + 1$, and g generates G, a cyclic subgroup of prime order q in Z_p^*:

$$G = \{g^0 = 1, g^1, g^2, \ldots, g^{q-1}\}$$

- NB: we can also define an analogous scheme *ECDH* over the (additive) group of points of an Elliptic Curve $G = \{O, P, 2P, 3P, \ldots, (r-1)P\}$ of order r

Diffie-Hellman Key Exchange

- In Diffie and Hellman's original presentation, we have a collection of *n* users
- All users make use of the same set of public parameters (p, q, g)
- Each of the *n* users U_i picks x_i uniformly at random from {0,1,…,*q*-1}
- User U_i's private key is x_i; its public key is $Y_i = g^{x_i} \bmod p$
- All the public keys Y_1 , …, Y_i , …, Y_n are assumed to be available in a public, authentic database
- To set up a shared, symmetric key between user U_i and user U_j:
  - User U_i computes $K_{ij} = (Y_j)^{x_i} = g^{x_j x_i} = g^{x_i x_j} \bmod p$;
  - User U_j computes $K_{ji} = (Y_i)^{x_j} = g^{x_i x_j} \bmod p$.
  - Critical point: the keys are the same for the two users!

Security of Diffie-Hellman Key Exchange

- Notice that **no communication** is needed between any pair of users in order to establish a key!
- Instead, the users are assumed to have access to the public, authentic database of public keys Y_1 , …, Y_i , …, Y_n
- An attacker can also learn public keys and would like to compute shared keys
- For example, it knows $Y_i = g^{x_i}$ mod *p* and $Y_j = g^{x_j}$ mod *p*, and would like to recover $K_{ij} = g^{x_i x_j}$ mod *p*
- The underlying algorithmic problem can be stated as follows:

  Given (*p*,*q*,*g*), and the values g^a mod *p*, g^b mod *p*, find g^{ab} mod *p*

- This is known as the **Computational Diffie-Hellman Problem** (CDHP)

Security of Diffie-Hellman Key Exchange

- The Computational Diffie-Hellman Problem (CDHP):

  Given (p,q,g), and the values $g^a \bmod p$, $g^b \bmod p$, find $g^{ab} \bmod p$

- If we can efficiently solve the DLP, then we can efficiently solve CDHP:
  - Find a from $g^a \bmod p$ by solving DLP, then compute $(g^b)^a \bmod p$
  - So CDHP is not harder than DLP
- We don't know in general whether CDHP is equivalent to DLP
  - This is widely believed to be so
  - This is known to be so in special cases
- This helps us choose secure parameters for Diffie-Hellman

A modern view of Diffie-Hellman Key Exchange

Pick random x_1;
$Y_1 = g^{x_1} \bmod p$

Y_1

Pick random x_2;
$Y_2 = g^{x_2} \bmod p$

Y_2

$K = (Y_1)^{x_2} \bmod p$

$K = (Y_2)^{x_1} \bmod p$

- It follows that parties compute the same *key* K
- In practice, we view K as a shared secret, from which the actual keys are derived
  - E.g., using a Key-Derivation Function (KDF)

A modern view of Diffie-Hellman Key Exchange

- The modern view of Diffie-Hellman key exchange regards the private values x_i and the public values $Y_i = g^{x_i} \bmod p$ as being ephemeral
  - That is, they are generated and used only once, setting up a fresh key each time
- Instead of looking up values in a public database:
  - Users first agree on parameters *g, p* and *q*
  - Then select fresh random private values x_i, x_j
  - Then exchange the corresponding public values Y_i , Y_j over a public channel
- It now seems that any two parties can securely agree upon a key without having had any previous association!

MITM attack on Diffie-Hellman Key Exchange

- An active man-in-the-middle attacker can modify the public values in transit!
- For example, the attacker can change Y_i to a value g^z mod p for which he knows z
- The attacker can then compute the key that user j would compute on receipt of g^z mod p, namely, $(g^z)^{x_j} = (g^{x_j})^z = (Y_j)^z$ mod p
- And similarly in the other direction
- This enables the attacker to convince Alice and Bob that they are talking to each other, while in fact they are both talking to the attacker!
- It also means that the attacker can decrypt and read the messages between Alice and Bob

MITM attack on Diffie-Hellman Key Exchange

Pick random z_1, z_2;
$Y_1' = g^{z_1} \bmod p$
$Y_2' = g^{z_2} \bmod p$

Pick random x_1;
$Y_1 = g^{x_1} \bmod p$

Pick random x_2;
$Y_2 = g^{x_2} \bmod p$

Y_1' ←

Y_1 ←

Y_2 →

Y_2' →

$K' = (Y_1')^{x_2} \bmod p = g^{z_1 x_2} \bmod p$

$K'' = (Y_2')^{x_1} \bmod p = g^{z_2 x_1} \bmod p$

Attacker can also compute

$K' = (Y_2)^{z_1} \bmod p = g^{x_2 z_1} \bmod p$

$K'' = (Y_1)^{z_2} \bmod p = g^{x_1 z_2} \bmod p$

An active attacker can completely compromise the security of "modern" Diffie-Hellman key exchange

This was not an issue in the original set-up: the public-key database was authentic

Preventing MITM attack on Diffie-Hellman Key Exchange

- The modern version requires the authenticity of the public values Y_i , Y_j
- How can this be done?
- Two solutions: use a MAC or use digital signatures

Q: If we had a key for a MAC, why do we need to agree a new key via Diffie-Hellman?

A: Forward security: even if the MAC key was later revealed, the new key from Diffie-Hellman would still be secure

Q: What additional requirements does the use of signatures bring?

A: We now need an assurance of the authenticity of the signature verification keys

- This is typically done using certificates, CAs, and PKI

Public-Key Encryption from discrete logs

From DH Key Exchange to PK Encryption

- Diffie-Hellman paper appeared in 1976

  https://www-ee.stanford.edu/~hellman/publications/24.pdf
  (see this week's *Further Reading & Research*)

  - Introduced the idea of public-key encryption
  - No concrete example of a public-key encryption scheme

Q: Can we get from DHKE to PKE?

A: Yes we can!

- We next define the ElGamal public-key encryption scheme from 1985

ElGamal Public-Key Encryption Scheme

Public parameters (p,q,g) as usual

KeyGen:

Pick x uniformly at random from {0,1,…,q-1}.

Set public key to be $X = g^x$ mod p and private key to be x

Enc: given public key X and message M (encoded as an element of G_q):

1. Pick r uniformly at random from {0,1,…,q-1}.
2. Set $Y = g^r$ mod p
3. Compute $Z = X^r$ mod p.
4. Output the ciphertext $C = (Y, M \cdot Z \bmod p)$.

Intuition: Encryption masks message M with Z (mask via multiplication, not XOR).
Y gives information to private key owner for computing Z

ElGamal Public Encryption Scheme

Dec: given private key x and ciphertext $C = (Y, C')$:

1. Check that Y is in G_q, return "fail" if not
2. Compute $Z' = Y^x \bmod p$
3. Output $M = C' \cdot (Z')^{-1} \bmod p$

Correctness of the scheme:

$$C' \cdot (Z')^{-1} = (M \cdot Z) \cdot (Z')^{-1} = (M \cdot X^r) \cdot (Y^x)^{-1} \mod p$$

$$= M \cdot (g^x)^r \cdot \left((g^r)^x\right)^{-1} \bmod p$$

$$= M \cdot g^{xr} \cdot (g^{xr})^{-1} \bmod p$$

$$= M$$

Security of ElGamal Public Encryption scheme

- IND-CPA security for ElGamal encryption can be proven based on a variant of the CDH Problem, called the Decisional Diffie-Hellman (DDH) Problem

DDH problem: Given (p,q,g),
distinguish the triple (g^a, g^b, g^{ab}) from the triple (g^a, g^b, g^c),
where a, b, c are uniformly random values in $\{0,1,\ldots,q\text{-}1\}$

Informal proof of security:

- Ciphertext includes $M \cdot Z$ mod p where $Z = g^{xr}$ mod p
- If (g^x, gr, g^{xr}) is indistinguishable from (g^x, gr, g^c), then we can "replace" $Z = g^{xr}$ by $Z = g^c$
- But, since g^c can take any value in G_q, it follows that $M \cdot Z$ mod p is uniformly random in G_q
- Hence M is *perfectly* hidden!

DHIES

Diffie-Hellman Integrated Encryption Scheme (DHIES)

- ElGamal has two significant disadvantages:
  - It is not IND-CCA secure
  - Messages need to be encoded as elements of G_q
    - Usually inconvenient to work with
- DHIES addresses both problems

DHIES

Public parameters (p,q,g) as usual, H a hash function

KeyGen:

Pick x uniformly at random from {0,1,…,q-1}
Set public key to be $X = g^x$ mod p and private key to be x

Enc: given public key X and message M (a bit-string):

1. Pick r uniformly at random from {0,1,…,q-1}
2. Set $Y = g^r$ mod p
3. Compute $Z = X^r$ mod p
4. Set $K = H(Z, X, Y)$
5. Split K into K_e and K_m
6. Compute C' = EtM(M) using keys K_e and K_m for encryption and MAC, respectively
7. Output the ciphertext $C = (Y, C')$

DHIES

Dec: given private key x and ciphertext C = (*Y, C'*)*:*

1. Check that *Y* is in G_q, return "fail" if not
2. Compute $Z = Y^x$ mod *p*
3. Set $K = H(Z, X, Y)$
4. Split K into K_e and K_m
5. Decrypt *C'* using keys K_e and K_m for encryption and MAC, respectively
6. Output "fail" if step 5 fails, otherwise output the message returned in step 5

Correctness relies on: $X^r = Z = Y^x$ mod *p*

- That is, we again implicitly have a Diffie-Hellman key exchange

DHIES

- DHIES is IND-CCA secure
- The security proof relies on
  - The hardness of a variant of the Computational Diffie-Hellman Problem
  - Assumption that H behaves like a random function
  - The symmetric encryption scheme is IND-CPA
  - The MAC scheme is UF-CMA
- DHIES an instance of a hybrid encryption scheme:
  - Send information Y that allows a shared key K to be computed
  - Use key K in a symmetric encryption scheme
- We could replace "EtM" with any AE scheme
- We have a combination of a Key Encapsulation Mechanism (KEM) and a Data Encapsulation Mechanism (DEM): the KEM/DEM paradigm

Elliptic curve cryptography

Recall: discrete logs over cyclic groups

- For a cyclic, finite (multiplicative) group G of order r generated by g, we defined the **Discrete Logarithm Problem (DLP) in *G*** as: given a randomly selected element *h* of $G = \{g^0, g^1, g^2, g^3, \ldots g^{r-1}\}$, find *x* in [*0,r-1*] such that $h = g^x$
- The first platform we defined to construct cryptographic schemes based on the DLP was the multiplicative group of non-zero integers modulo a prime *p*

$$Z_p^* = \{1, 2, \ldots, p-1\}$$

- We described how to select parameters to offer the desired level of security, based on the complexity of the main DLP-solving algorithms that apply to Z_p^*
- We took $p = kq + 1$, where *q* is also prime, and instantiated DL over the cyclic subgroup of Z_p^* of order *q*
- We now define another platform for constructing DLP-based crypto: Elliptic Curves

Elliptic Curves

- We have seen several fields: real numbers, rational numbers, Z_p
- An elliptic curve over a field F is a set of pairs $(x,y) \in F^2$ that satisfy the following equation

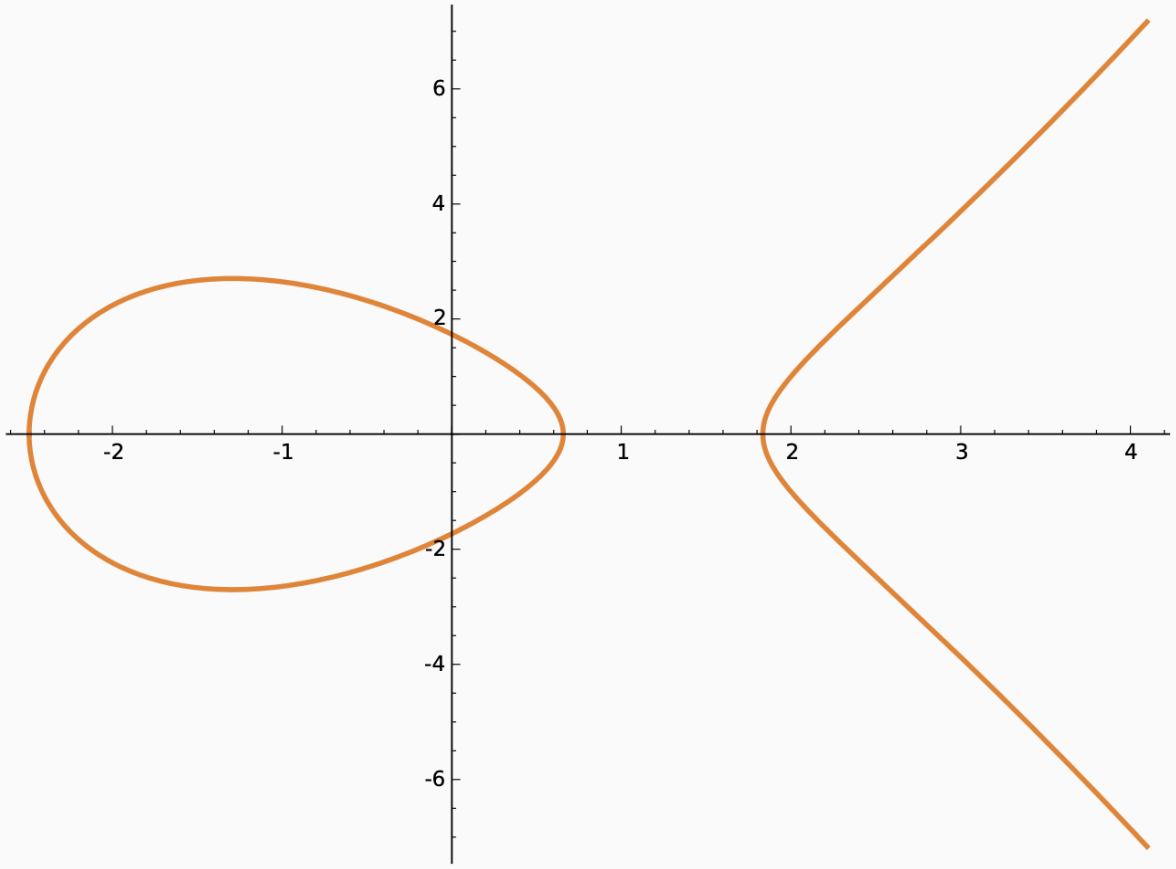
$$y^2 = x^3 + ax + b \qquad (\text{where } a,b \in F, \text{ and } 4a^3 + 27b^2 \neq 0),$$

plus a special "point at infinity" O

- Using different fields F and different values for a,b give different curves
- We denote elliptic curve as $E = E(F) = \{(x,y) \in F^2 \mid y^2 = x^3 + a.x + b\} \cup \{O\}$

An elliptic curve over the real numbers

$$y^2 = x^3 - 5x + 3$$




```
sage: E = EllipticCurve([-5, 3])
```

An elliptic curve over Z_5

$$y^2 = x^3 + 2x + 4 \pmod 5$$

x	0	1	2	3	4
x^3	0	1	3	2	4
$2x$	0	2	4	1	3
4	4	4	4	4	4
y^2	4	2	1	2	1
y	2,3		1,4		1,4

E has 7 points: $0, (0,2), (0,3), (2,1), (2,4), (4,1), (4,4)$

```
sage: E = EllipticCurve(GF(5), [2, 4])
```

Example: elliptic curve over Z_{97}

$$y^2 = x^3 - x + 1 \pmod{97}$$

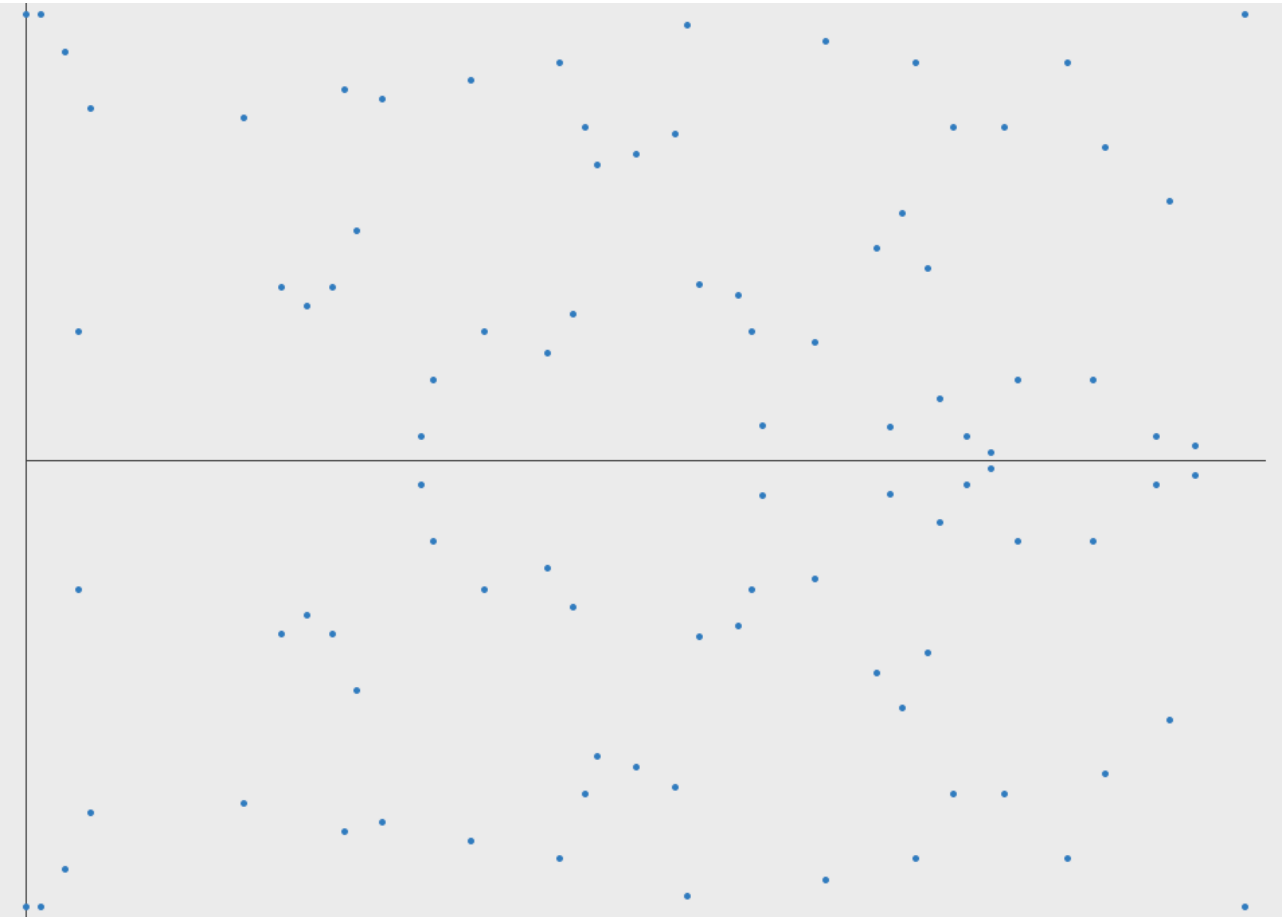


figure source: https://blog.cloudflare.com/a-relatively-easy-to-understand-primer-on-elliptic-curve-cryptography/

The group of points of an elliptic curve

- We can define an **addition operation** over the points of an elliptic curve

$$E = \{(x,y) \in F^2 \mid y^2 = x^3 + a.x + b\} \cup \{O\}$$

- Points $P = (x,y)$ on E can be added to obtain another point on the curve
- In fact, under this addition operation, **E is an additive group**!
- The point at infinity O is the (additive) identity of this group
  - $P + O = O + P = P$ for all elliptic curve points P
- Each point P has an (additive) inverse: $-P$
  - if $P = (x,y)$ then $-P = (x,-y)$
  - $P + (-P) = O$
  - Point at infinity O is its own inverse

Elliptic curve group: addition operation

- The addition operation is **not** simply adding the coordinates of P and Q
- It is more complicated:

Adding points (x_1, y_1) **and** (x_2, y_2)	**Doubling point** (x, y)
1. $\lambda = (y_2 - y_1)/(x_2 - x_1)$	1. $\lambda = (3x^2 + a)/(2y)$
2. $x_3 = \lambda^2 - x_1 - x_2$	2. $x' = \lambda^2 - 2x$
3. $y_3 = \lambda(x_1 - x_3) - y_1 = \lambda(x_2 - x_3) - y_2$	3. $y' = \lambda(x - x') - y$
4. return (x_3, y_3)	4. return (x', y')

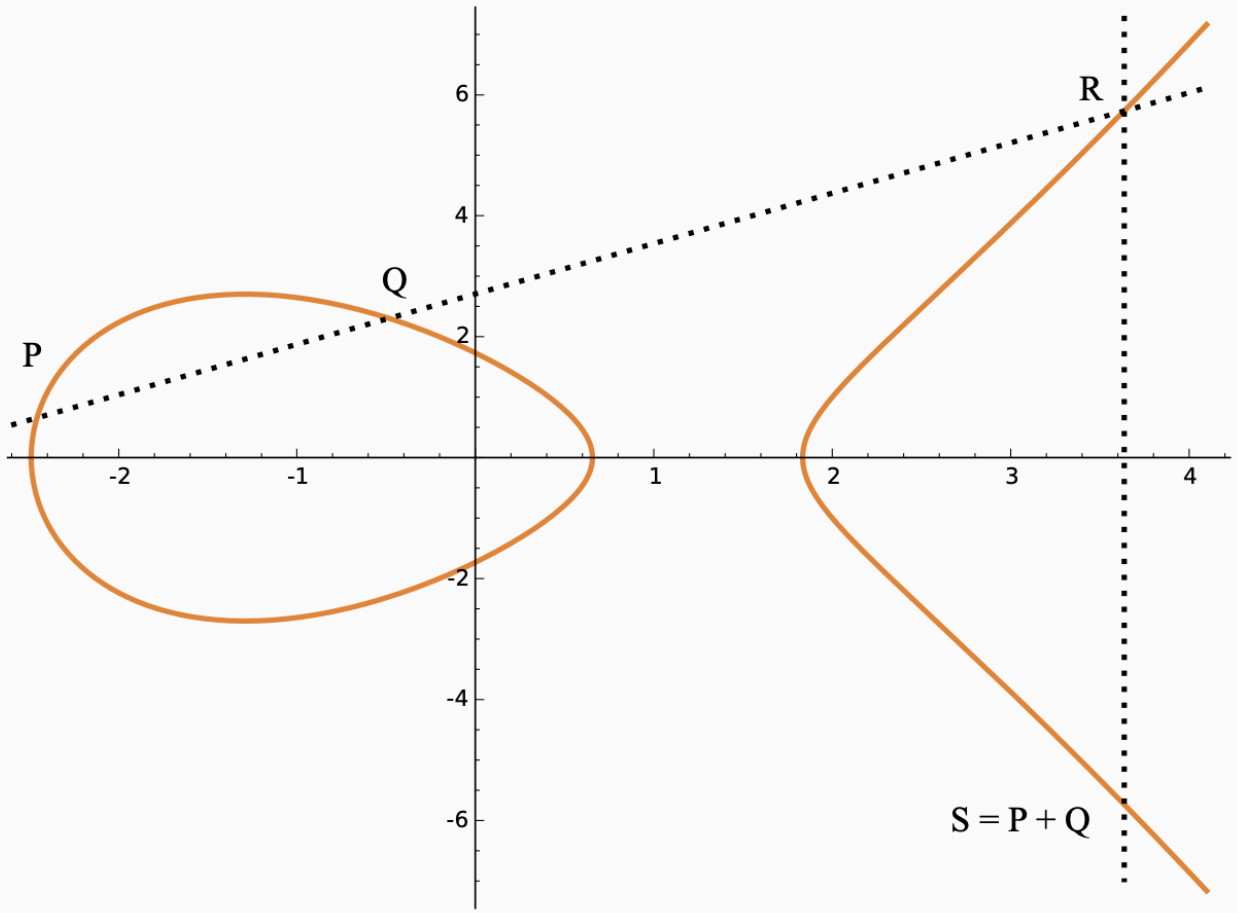
- **Notation**: since the group is additive, with elliptic curves we write

$$Q = (P + P + \ldots + P) = k.\ P$$

rather than $Q = (P\ .\ P\ .\ \ldots\ .\ P) = P^k$ as we did with the multiplicative group Z_p^*

Geometric interpretation over the reals

- $P + Q + R = O$
- $S = P + Q = -R$



Geometric interpretation over a finite field

- $y^2 = x^3 - x + 1 \pmod{97}$

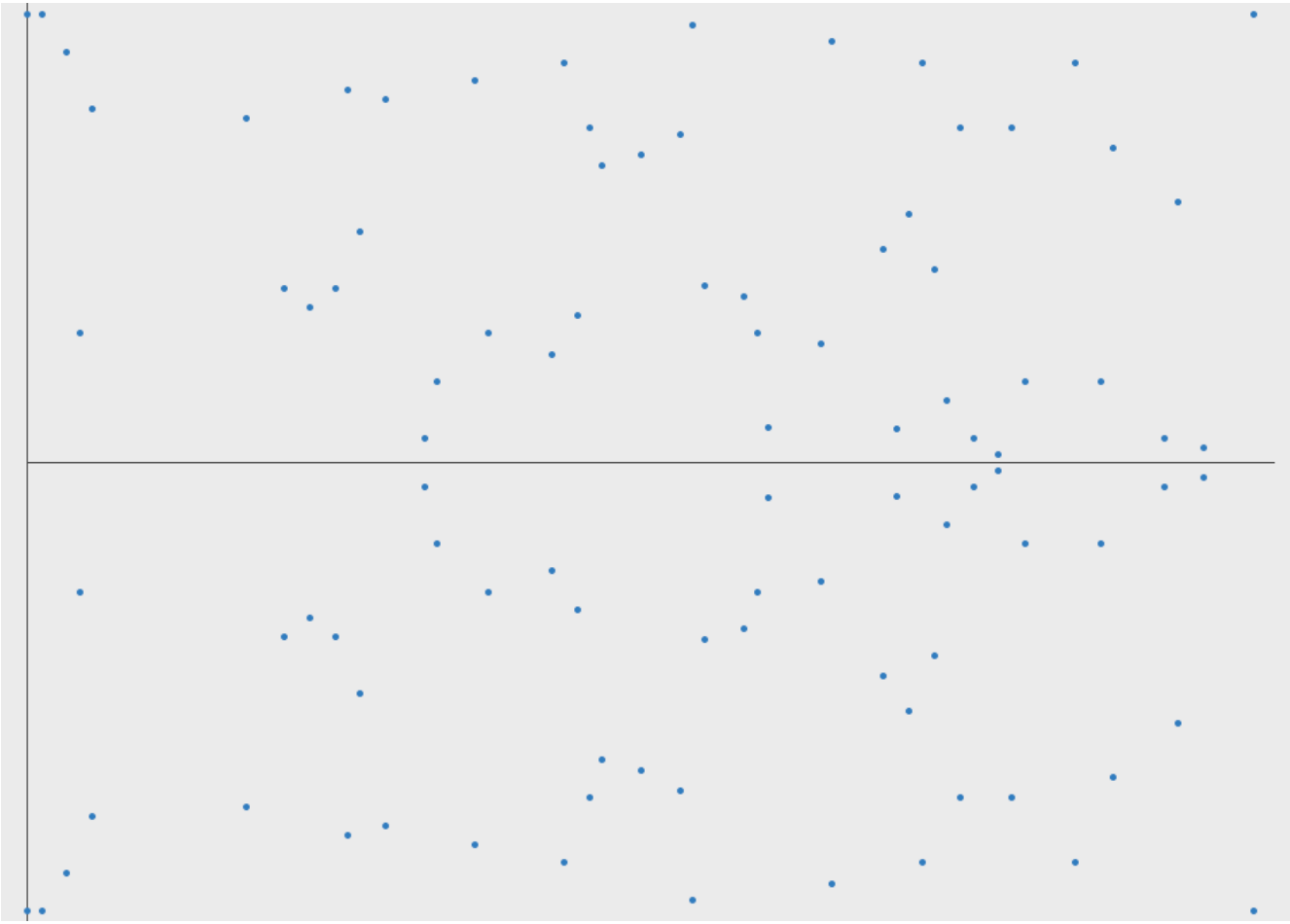


figure source: https://blog.cloudflare.com/a-relatively-easy-to-understand-primer-on-elliptic-curve-cryptography/

Properties of the elliptic curve group

For curves over a finite field $F = Z_p$, where p is prime we have:

- The order r of E(F) satisfies $p + 1 - 2\sqrt{p} \leq r \leq p + 1 + 2\sqrt{p}$
  - i.e., the order of E(F) is close to p
- The group E(F) is either cyclic or the product of two cyclic (sub)groups
- We will be working over cyclic (sub)groups of E(F) of large order

Example: E = $\{(x,y) \mid y^2 = x^3+2x+4\}$ over Z_5 has 7 points (thus it is cyclic)

- $P = (0,2)$ is a generator of E
- The correspondence between points and multiples kP is unclear
- This is the basis for the security of elliptic curve cryptosystems

Elliptic Curve Discrete Logarithm Problem (ECDLP)

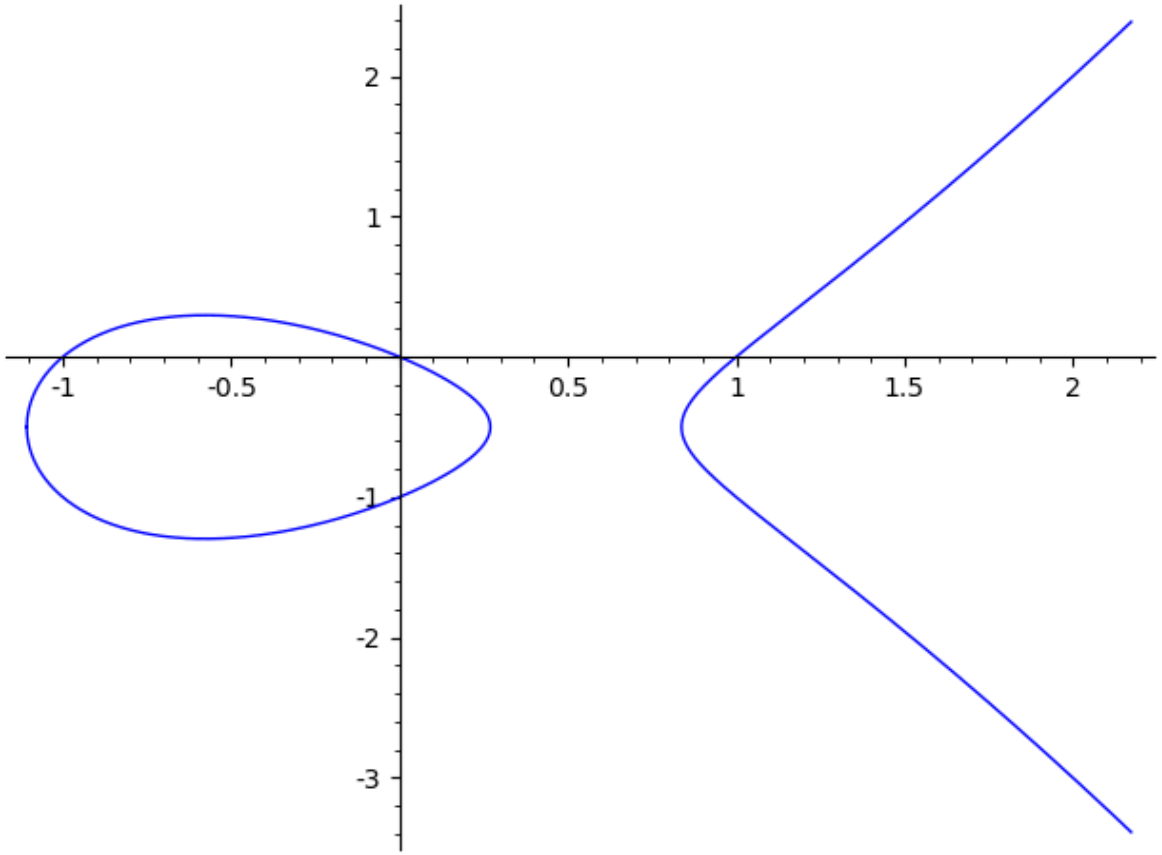
Let G be a cyclic subgroup of order r of the group of points of an elliptic curve E generated by the point P, i.e.

$$G = \{ P, 2P, 3P, 4P, \ldots, (r-1)P, O\}$$

Elliptic Curve Discrete Logarithm Problem (ECDLP) in *G*:

Given randomly chosen point Q in G, find k such that $Q = kP$

Elliptic curve trapdoor illustrated



figure/animation source: https://medium.com/@icostan/animated-elliptic-curves-cryptography-122fff8fcae

Elliptic Curve Cryptography (ECC)

- ECC replaces the finite multiplicative group of integers modulo a prime p with an elliptic curve group
- Instead of multiplication modulo p, the group operation is addition of curve points (i.e., $P+Q$)
- The analogue of exponentiation modulo p is computing scalar multiples of curve points (i.e., kP)
- The security of ECC schemes depends on hardness of ECDLP

Why ECC?

- Currently there are no sub-exponential algorithms for solving ECDLP
  - Index Calculus does not apply to Elliptic curves!
  - The best algorithms run in square-root time (e.g., Pollard's rho)
- This allows us to work with smaller parameters
  - We will use curves over a prime field p which give rise to a cyclic group of curve points, of order r close to p.
  - The best attack will then have complexity O($\sqrt{r}$)
  - So $p \approx r$ of 256 bits gives approximately 128 bits of security
- In turn this allows more efficient implementations
- We can define analogous ECC schemes for DH, ElGamal,…

Set-up of a system to use ECC

- We need to decide on a field *F*
  - Specifically, a finite field Z_p for a prime *p*
- We need to decide on a curve *E*
- We need to find a base point *P*
  - The generator of the cyclic group of *E*, of large, known order
- We need to support the new arithmetic

In practice you will use a standardised curve!

ECC: standardised curves

Standards (e.g. NIST, SECG, ANSI, Brainpool, CFRG) specify:

- Which fields F; coefficients a, b; and base points P to use
  - e.g., ‘secp160r1’, ‘nistp256’
- How to do point multiplication
  - Given k,P, how to compute $kP = P + \cdots + P$
- Representation of points (x,y) on the wire, point compression, etc
- Plus, a huge amount of possible optimisations

ECC: summary

- ECC gives alternative way to set up DLP-based cryptographic schemes
- ECC allows for smaller parameters and more efficient implementations
- **Do not make your own curves and/or points, but use standards**
  - NIST curves are widely supported, e.g., NIST P-256 or NIST P-384
  - Curve 25519 is seeing increasing adoption for its implementation benefits
  - If you want to learn more about these curves, see https://safecurves.cr.yp.to

Homework

- Read Chapters 5 and 9 of "*Everyday Cryptography*"
- Read Chapters 9, 11, 12 of "*Serious Cryptography*"
- Check out the *Further Reading & Research* on Moodle